

UNIVERSITÀ DEGLI STUDI DI PAVIA
CORSO DI INGEGNERIA DEL SOFTWARE

Specifica dei Requisiti Software

PM_Collegio : Sistema di gestione prenotazioni camere
per collegi universitari

Progetto	PM_Collegio
Materia	Ingegneria del Software, Università degli Studi di Pavia
Autore	Giresse Kengne
Versione documento	1.0
Data	giugno 2026

1. Introduzione

1.1 Scopo del documento

Questo documento costituisce la **Specifica dei Requisiti Software** (Software Requirements Specification, SRS) del sistema **PM_Collegio**. Il suo scopo è :

- definire in modo completo, univoco e verificabile i requisiti funzionali, non funzionali e di interfaccia del sistema ;
- fornire una base condivisa tra committente (il collegio universitario) e gruppo di sviluppo per la validazione del prodotto ;
- servire da riferimento per le attività di progettazione, implementazione, testing e manutenzione ;
- documentare i vincoli architetturali e tecnologici adottati.

Il documento è destinato a : docente e commissione del corso di Ingegneria del Software, membri del gruppo di sviluppo, eventuali manutentori futuri del sistema.

1.2 Scopo del prodotto

PM_Collegio è un'applicazione desktop per la **gestione delle prenotazioni di camere in collegi universitari**. Il prodotto permette di :

- gestire l'intero ciclo di vita di una prenotazione : creazione tramite check-in, modifica con tracciamento storico, chiusura tramite check-out con emissione automatica della fattura ;
- amministrare le entità del dominio : camere, utenti, committenti (collegi) ;
- gestire la fatturazione e i pagamenti simulati (carta, PayPal, bonifico) ;
- autenticare gli utenti con gestione di sessione e ripristino password tramite domanda di sicurezza ;
- monitorare gli accessi al sistema a fini di audit.

Lo schema dati è progettato per la **multi-committenza** (più collegi sulla stessa installazione), ma il perimetro della consegna si limita all'esercizio del sistema per **un solo committente**.

Il prodotto è l'evoluzione architetturale del prototipo monolitico v1 : ristrutturato secondo il pattern **MVC + DAO + DTO** per migliorare manutenibilità, testabilità e separazione delle responsabilità.

1.3 Definizioni, acronimi e abbreviazioni

Termine	Definizione
AS	Admin di Sistema : amministratore globale dell'installazione
AC	Admin Committente : amministratore di un singolo collegio
AR	Addetto alla Reception (receptionist)
U	Utente cliente (studente/ospite del collegio)

Committente	Il collegio universitario cliente dell'installazione (entità <code>Committente</code>)
Check-in	Operazione di apertura di una prenotazione e assegnazione della camera
Check-out	Operazione di chiusura della prenotazione con calcolo dell'importo e fatturazione
Fattura	Documento contabile generato dal sistema, con stati <code>pagato</code> , <code>non pagato</code> , <code>in attesa</code>
user_counter	Identificativo applicativo univoco dell'utente, formato <code>XXX{id}{NOME}</code> (VARCHAR, non autoincrementale)
Sessione	Contesto di autenticazione con token UUID a 64 caratteri e scadenza a 8 ore
MVC	Model-View-Controller, pattern architetturale di separazione tra presentazione, logica e dati
DAO	Data Access Object, classe che incapsula l'accesso al database per una tabella
DTO	Data Transfer Object, oggetto read-only che trasporta dati joinati tra layer
SRS	Software Requirements Specification, il presente documento
JFrame	Finestra dell'interfaccia grafica Java Swing
JDBC	Java Database Connectivity, API standard Java per l'accesso a database relazionali
CRUD	Create, Read, Update, Delete : le quattro operazioni base sui dati

Convenzione identificativi dei requisiti : **RF-n** requisito funzionale, **RNF-n** requisito non funzionale, **RI-n** requisito di interfaccia.

1.4 Riferimenti

1. **ReadMe.md** : specifica redatta dal gruppo, documento sorgente dei requisiti.
2. **DOCUMENTAZIONE** : documentazione tecnica del progetto PM_Collegio.
3. **SPECIFICA_REQUISITI.pdf** : Specifica dei Requisiti Software.
4. **PROGETTAZIONE**, Documento di Progettazione.
5. **Schema database MySQL** : script DDL con dati di seed (utenti, indirizzi, committenti).
6. Codice sorgente di riferimento : repository Git di `PM_CollegioV2` , package `it.collegio.*` .

1.5 Descrizione del resto del documento

- Il **Capitolo 2** fornisce una descrizione generale del prodotto : il suo posizionamento, le macro-funzioni, le classi di utenti, i vincoli e i presupposti.
- Il **Capitolo 3** elenca i requisiti specifici, suddivisi in requisiti funzionali (3.1), non funzionali (3.2) e di interfaccia (3.3), ciascuno con identificativo univoco per la tracciabilità.
- Il **Capitolo 4** contiene le appendici : schema dati di riferimento e matrice di tracciabilità requisiti-componenti.
- Il **Capitolo 5** riporta l'indice del documento.

2. Descrizione generale

2.1 Prospettiva del prodotto

PM_Collegio è un **sistema autonomo** (standalone) : non si integra con sistemi esterni preesistenti. Le sue uniche dipendenze esterne sono :

- un **DBMS MySQL** che ospita la base di dati relazionale ;
- la **Java Runtime Environment** sulla postazione desktop dell'utente.

Il sistema è strutturato in tre strati logici, sullo stesso processo applicativo :

Presentation	(views, Swing)
Application	(controllers)
Data Access	(dao, dto, models)
Database	(MySQL, JDBC)

Una View non accede mai direttamente al database : ogni operazione passa da un Controller, che orchestra uno o più DAO. I pagamenti sono **simulati** : vengono persistiti su database ma non esiste alcuna integrazione con gateway di pagamento reali.

2.2 Funzioni del prodotto

1. **Autenticazione e gestione sessione** : login con email e password, logout, sessione con token a scadenza, ripristino password tramite domanda di sicurezza, registrazione di nuovi clienti.
2. **Gestione prenotazioni** : visualizzazione (filtrata per ruolo), ricerca, modifica con registrazione dello storico.
3. **Check-in** : creazione transazionale di prenotazione, fattura iniziale e occupazione della camera.
4. **Check-out** : chiusura transazionale della prenotazione, liberazione della camera, calcolo dell'importo e aggiornamento della fattura.
5. **Fatturazione e pagamenti** : consultazione fatture, pagamento simulato con scelta del metodo, annullamento, regola di pagabilità combinata fattura + prenotazione.
6. **Gestione anagrafiche** : CRUD su camere, utenti e committenti.
7. **Monitoraggio e audit** : log degli accessi (login/logout, IP, note) e storico delle modifiche alle prenotazioni.

2.3 Caratteristiche utente

Ruolo	Codice	Profilo atteso	Operazioni principali
-------	--------	----------------	-----------------------

Admin di sistema	AS	Tecnico, gestisce l'installazione	Gestione committenti, utenti, camere, configurazione globale, log
Admin committente	AC	Gestore del collegio, competenza gestionale	Gestione camere, utenti, prenotazioni e fatture del proprio collegio
Receptionist	AR	Operatore di front-office, formazione minima	Check-in, check-out, gestione prenotazioni
Cliente	U	Studente/ospite, nessuna competenza tecnica	Consultazione delle proprie prenotazioni (sola lettura), pagamento e annullamento delle proprie fatture

L'interfaccia deve quindi essere utilizzabile senza formazione specifica : le funzioni visibili dipendono dal ruolo (la Home mostra solo i bottoni pertinenti) e le viste condivise si adattano al ruolo dell'utente connesso.

2.4 Vincoli generali

- **VG-1. Stack tecnologico** : Java SE con interfaccia Swing (JFrame), database MySQL, build Ant generata da NetBeans (`build.xml`).
- **VG-2. Nessun framework esterno** : il progetto è didattico e non utilizza framework di terze parti (no Spring, no Hibernate, no librerie ORM). L'accesso ai dati è JDBC puro.
- **VG-3. Architettura imposta** : pattern MVC + DAO + DTO, package `it.collegio.<layer>` . Le query SQL sono centralizzate in `utilities/QueryContainer.java` .
- **VG-4. Perimetro mono-committente** : lo schema supporta più committenti, ma la consegna copre l'esercizio di un solo committente (id committente di default = 1).
- **VG-5. Pagamenti simulati** : nessuna integrazione con sistemi di pagamento reali.
- **VG-6. Applicazione desktop** : nessun requisito di accesso remoto o concorrenza distribuita per la v2 (il front-end web è oggetto della v3, fuori dal perimetro di questo documento).
- **VG-7. Credenziali esternalizzate** : i parametri di connessione al database risiedono in `db.properties` , non versionato (`.gitignore`), letto da `DatabaseConnection` via classpath.
- **VG-8. Scadenza di consegna** : giugno 2026.

2.5 Presupposti e dipendenze

- **P-1.** Sulla postazione è installata una JRE/JDK compatibile con il progetto NetBeans.
- **P-2.** Il DBMS MySQL è raggiungibile con i parametri indicati in `db.properties` e lo schema è stato creato con lo script DDL di riferimento, inclusi i **dati di seed** (utenti, mansioni, indirizzi, committenti, metodi di pagamento) necessari al primo avvio.
- **P-3.** Esiste almeno un committente con id 1 : la registrazione self-service dei clienti vi associa i nuovi utenti.
- **P-4.** Esiste almeno un indirizzo con id 1, usato come fallback quando l'indirizzo scelto in registrazione non è risolvibile.
- **P-5.** La mansione con id 4 corrisponde al ruolo cliente (U) : è il ruolo assegnato di default in registrazione.

- **P-6.** L'orologio di sistema della postazione è affidabile : scadenza sessione e date di emissione fattura si basano su di esso.
- **P-7.** L'applicazione è usata da un operatore alla volta per postazione ; la concorrenza tra postazioni è gestita dal DBMS.

3. Requisiti specifici

3.1 Requisiti funzionali

3.1.1 Autenticazione e sessione

ID	Requisito
RF-1	Il sistema deve permettere il login tramite email e password. Se le credenziali non sono valide, il sistema mostra l'errore "Credenziali errate" e non concede l'accesso.
RF-2	Il login deve essere concesso solo a utenti in stato <code>attivo</code> . Gli utenti in stato <code>attesa</code> o <code>disattivato</code> ricevono il messaggio "Account non attivo o in attesa di approvazione".
RF-3	A ogni login riuscito il sistema deve registrare un record in <code>AccessLog</code> con utente, indirizzo IP, nota e timestamp di login.
RF-4	A ogni login riuscito il sistema deve creare una sessione applicativa con token univoco (UUID, 64 caratteri) e scadenza a 8 ore dalla creazione, persistita nella tabella <code>sessions</code> .
RF-5	Dopo il login, il contesto di sessione (<code>SessionContext</code>) deve contenere : <code>user_counter</code> , email, nome, cognome, id log, id sessione, committente, ruolo (id, nome, tipo) e token.
RF-6	Al logout il sistema deve aggiornare <code>AccessLog.logout_time</code> , invalidare la sessione su database e svuotare il contesto di sessione.
RF-7	Il sistema deve permettere la registrazione self-service di un nuovo cliente con : nome, cognome, email, password, telefono, indirizzo, domanda e risposta di sicurezza, genere. Nome, email e password sono obbligatori.
RF-8	Alla registrazione, il nuovo utente deve essere creato con ruolo cliente (U), committente di default e stato <code>attesa</code> (richiede attivazione da parte di un amministratore prima di poter accedere).
RF-9	Il sistema deve rifiutare la registrazione se l'email è già in uso. L'email viene normalizzata in minuscolo prima del salvataggio.
RF-10	Il sistema deve generare per ogni nuovo utente uno <code>user_counter</code> univoco nel formato <code>XXX{id}{NOME}</code> , con retry in caso di collisione.
RF-11	Il sistema deve permettere il ripristino della password : dato l'indirizzo email, mostra la domanda di sicurezza dell'utente ; se la risposta fornita coincide con quella registrata, consente di impostare una nuova password. In ogni altro caso il ripristino fallisce senza rivelare informazioni.

3.1.2 Prenotazioni

ID	Requisito
RF-12	Il sistema deve mostrare l'elenco delle prenotazioni con i dettagli di utente e camera (DTO joinato). Per il ruolo U l'elenco è filtrato server-side alle sole prenotazioni dell'utente connesso.
RF-13	Il sistema deve permettere la ricerca di una prenotazione per identificativo. Per il ruolo U la ricerca restituisce risultati solo tra le proprie prenotazioni.

RF-14	La selezione di una riga nella tabella delle prenotazioni deve popolare automaticamente il form di dettaglio, senza dover digitare l'ID.
RF-15	Il sistema deve permettere la modifica delle date (check-in, check-out) e delle note di una prenotazione. L'aggiornamento ricalcola i giorni server-side.
RF-16	Ogni modifica a una prenotazione deve registrare in <code>StoricoPrenotazioni</code> le date precedenti e le nuove, l'utente e il timestamp di modifica. Update e inserimento storico avvengono nella stessa transazione : se uno dei due fallisce, nessuna modifica è persistita.
RF-17	Per il ruolo U la vista prenotazioni è in sola lettura : il bottone Update non è disponibile e i campi data/note non sono editabili.
RF-18	Gli identificativi di prenotazione devono essere presentati in formato alfanumerico <code>P0nnn</code> (es. <code>P0042</code>) ; quelli delle camere in formato <code>R0nnn</code> .

3.1.3 Check-in

ID	Requisito
RF-19	Il sistema deve mostrare le sole camere in stato <code>disponibile</code> , con possibilità di filtro per prezzo.
RF-20	Il sistema deve permettere la ricerca del cliente per email ai fini dell'assegnazione della prenotazione.
RF-21	Il check-in deve eseguire in un'unica transazione : (1) inserimento della <code>Reservation</code> in stato <code>attiva</code> con data di check-in e note ; (2) creazione della <code>Fattura</code> associata con importo 0 e stato <code>in attesa</code> ; (3) aggiornamento della camera a stato <code>occupata</code> . Se uno qualunque dei passi fallisce, l'intera operazione viene annullata (rollback).
RF-22	Una data di check-in non valida o non parsabile (formato atteso <code>yyyy-MM-dd</code>) deve far fallire l'operazione senza effetti sul database.

3.1.4 Check-out

ID	Requisito
RF-23	Prima della conferma, il sistema deve mostrare un'anteprima del check-out con : dati del cliente, camera, prezzo giornaliero, date, numero di giorni e importo totale calcolato.
RF-24	Il numero di giorni deve essere calcolato come differenza tra data di check-out e data di check-in ; un soggiorno con date coincidenti vale 1 giorno (minimo fatturabile).
RF-25	L'importo della fattura deve essere ricalcolato server-side come <code>prezzo_giornaliero × giorni</code> ; il sistema non si fida dei valori mostrati nella vista.
RF-26	Il check-out deve eseguire in un'unica transazione : (1) aggiornamento della <code>Reservation</code> a stato <code>completata</code> con data di check-out e giorni ; (2) aggiornamento della camera a stato <code>disponibile</code> ; (3) emissione o aggiornamento (upsert) della <code>Fattura</code> con l'importo calcolato e stato <code>in attesa</code> . Rollback completo in caso di errore.

3.1.5 Fatture e pagamenti

ID	Requisito
RF-27	Il sistema deve mostrare l'elenco delle fatture con dettagli della prenotazione associata, incluso lo stato della prenotazione (colonna "Stato Prenot."). Per il ruolo U l'elenco è filtrato server-side alle sole fatture dell'utente connesso.
RF-28	Il sistema deve permettere il pagamento di una fattura scegliendo il metodo tra quelli censiti (carta, PayPal, bonifico). Il pagamento è simulato : viene registrato un record <code>Pagamento</code> con importo e data, e la fattura passa a stato <code>pagato</code> .
RF-29	Il sistema deve permettere l'annullamento di una fattura, che la porta in stato <code>non pagato</code> .
RF-30	Regola di pagabilità combinata : i bottoni Paga e Annulla devono essere abilitati solo se la fattura è in stato <code>in attesa</code> o <code>non pagato</code> e la prenotazione correlata è in stato <code>completata</code> o <code>cancellata</code> . In tutti gli altri casi i bottoni sono disabilitati.

3.1.6 Gestione anagrafiche

ID	Requisito
RF-31	Il sistema deve permettere agli amministratori la gestione CRUD delle camere : numero, tipo (<code>singola</code> , <code>doppia</code> , <code>suite</code>), prezzo, tipo letto (<code>matrimoniale</code> , <code>singolo</code> , <code>king-size</code>), stato (<code>disponibile</code> , <code>occupata</code> , <code>manutenzione</code>).
RF-32	Il sistema deve permettere agli amministratori la gestione CRUD degli utenti : anagrafica, ruolo, committente, stato (<code>attivo</code> , <code>disattivato</code> , <code>attesa</code>), contatti, domanda/risposta di sicurezza.
RF-33	Il sistema deve permettere all'admin di sistema la gestione CRUD dei committenti : ragione sociale, gestore, contatti, indirizzo.
RF-34	Nelle viste di gestione (utenti, committenti, camere), la selezione di una riga deve attivare la modalità modifica : bottone Add disabilitato, bottone Update abilitato (mutua esclusione Add/Update).

3.1.7 Monitoraggio e controllo accessi

ID	Requisito
RF-35	Il sistema deve mettere a disposizione una vista Logs con l'elenco degli accessi : utente, IP, note, orario di login e di logout.
RF-36	La Home deve mostrare esclusivamente le funzioni consentite al ruolo dell'utente connesso (<code>configureButtonsForRole</code>), secondo la matrice ruoli/operazioni del §2.3.

3.2 Requisiti non funzionali

3.2.1 Architettura e manutenibilità

ID	Requisito
----	-----------

RNF-1	Il sistema deve rispettare l'architettura a strati MVC + DAO + DTO : nessuna View deve contenere SQL o accedere direttamente al database ; ogni accesso passa da Controller e DAO.
RNF-2	Tutte le query SQL devono essere centralizzate nella classe <code>QueryContainer</code> , nessuna query inline nei DAO.
RNF-3	Le entità di dominio (package <code>models</code>) devono essere POJO in corrispondenza 1:1 con le tabelle del database ; i dati joinati viaggiano in DTO read-only dedicati.
RNF-4	I nomi delle classi devono rendere esplicito il layer di appartenenza tramite i suffissi <code>View</code> , <code>Controller</code> , <code>Dao</code> .

3.2.2 Integrità dei dati

ID	Requisito
RNF-5	Le operazioni multi-tabella critiche (login con log + sessione, check-in, check-out, aggiornamento prenotazione con storico) devono essere atomiche : commit solo se tutti i passi riescono, rollback altrimenti.
RNF-6	I valori calcolati (giorni di soggiorno, importi fattura) devono essere determinati dal layer applicativo server-side, mai accettati così come presentati dall'interfaccia.
RNF-7	I filtri di visibilità per ruolo (prenotazioni e fatture del cliente U) devono essere applicati a livello di query , non solo nascondendo elementi dell'interfaccia.

3.2.3 Sicurezza

ID	Requisito
RNF-8	Le credenziali di connessione al database non devono comparire nel codice sorgente né nel repository : risiedono in <code>db.properties</code> , escluso dal versionamento.
RNF-9	Le sessioni devono scadere automaticamente dopo 8 ore dalla creazione ; il logout deve invalidare la sessione attiva.
RNF-10	Il flusso di ripristino password non deve rivelare se un'email esiste o se la risposta di sicurezza è errata : in caso di fallimento il sistema restituisce un esito negativo generico.
RNF-11	Ogni accesso (riuscito) al sistema deve essere tracciato in <code>AccessLog</code> a fini di audit.

3.2.4 Usabilità

ID	Requisito
RNF-12	I campi email e telefono devono essere validati live durante la digitazione nelle viste Login, Registrazione, Check-in e Ripristino password (email : formato standard ; telefono : esattamente 10 cifre).
RNF-13	All'avvio il sistema deve mostrare una schermata di caricamento (splash con barra di progresso) prima della schermata di login.

RNF-14	I messaggi di errore devono essere espressi in linguaggio comprensibile all'utente finale (es. "Email già in uso", "Credenziali errate").
---------------	---

3.2.5 Portabilità e vincoli tecnologici

ID	Requisito
RNF-15	Il sistema deve essere eseguibile su qualunque piattaforma dotata di JRE compatibile (Windows, Linux, macOS) senza modifiche al codice.
RNF-16	Il build deve avvenire tramite Ant (<code>build.xml</code> NetBeans) senza dipendenze da librerie esterne oltre al connettore JDBC MySQL.
RNF-17	Lo schema dati deve mantenere il supporto multi-committenza (chiave <code>committente_id</code> sulle entità pertinenti) anche se l'esercizio è mono-committente.

3.3 Requisiti di interfaccia

3.3.1 Interfacce utente

ID	Requisito
RI-1	L'interfaccia utente è realizzata con Java Swing : una finestra <code>JFrame</code> per ciascuna vista funzionale (Loading, Login, Registration, PassWord, Home, ManageReservation, ManageRoom, ManageUser, ManageTenant, CheckIn, CheckOut, Fattura, GestioneFatture, Logs).
RI-2	Gli elenchi sono presentati in tabelle (<code>JTable</code>) con selezione di riga che popola il form di dettaglio associato.
RI-3	Le date sono inserite e visualizzate nel formato <code>yyyy-MM-dd</code> .
RI-4	La Home adatta dinamicamente i bottoni visibili al ruolo dell'utente connesso.

3.3.2 Interfacce software

ID	Requisito
RI-5	Il sistema comunica con il database MySQL tramite JDBC ; i parametri di connessione (URL, utente, password) sono letti dal file <code>db.properties</code> sul classpath, all'avvio.
RI-6	Non sono previste interfacce verso sistemi esterni : i pagamenti non comunicano con gateway reali.

3.3.3 Interfacce hardware e di comunicazione

ID	Requisito
RI-7	Il sistema richiede una postazione desktop standard con tastiera, mouse e display ; nessun hardware dedicato.

RI-8	La comunicazione applicazione-database avviene su rete TCP/IP tramite il protocollo del connettore MySQL ; nessun altro protocollo di rete è richiesto dalla v2.
-------------	--

4. Appendici

Appendice A : Schema dati di riferimento (MySQL)

```
Indirizzo (indirizzo_id, paese, provincia, citta, via, cap)
Mansione (role_id, role_type, role_nome)
Committente (codCommittente, ragione_Sociale, gestore→User, email, telefono,
             indirizzo_id→Indirizzo)
User (user_counter PK VARCHAR, nome, cognome, email, access [password],
      ruolo→Mansione, committente_id→Committente,
      stato ENUM(attivo/disattivato/attesa),
      telefono, indirizzo_id→Indirizzo, recupero [domanda],
      response [risposta], genere)
Room (room_id, committente_id→Committente, numero_stanza,
      tipo ENUM(singola/doppia/suite), prezzo,
      letto_tipo ENUM(matrimoniale/singolo/king-size),
      stato ENUM(disponibile/occupata/manutenzione))
Reservation (reservation_id, user_id→User, committente_id→Committente,
             room_id→Room, check_in, check_out, giorni,
             status ENUM(attiva/cancellata/completata), note)
Fattura (fattura_id, reservation_id→Reservation, importo, data_emissione,
        stato ENUM(pagato/non pagato/in attesa))
MetodoPagamento (metodo_id, nome, descrizione)
Pagamento (pagamento_id, fattura_id→Fattura, metodo_id→MetodoPagamento,
            importo, data_pagamento)
AccessLog (log_id AUTO_INCREMENT, user_id→User, ip_address, note,
           login_time DEFAULT NOW(), logout_time NULL)
sessions (session_id, user_id→User, committente_id→Committente,
          role_id→Mansione, token CHAR(64), expires DATETIME, created_at)
StoricoPrenotazioni (storico_id, reservation_id→Reservation, user_id→User,
                    check_in_precedente, check_out_precedente,
                    nuovo_check_in, nuovo_check_out, data_modifica)
```

Nota : User.user_counter è una VARCHAR(100) (es. XXXADMIN , XXX42MARIO), non un intero autoincrementale ; la generazione è applicativa (UserDao.generateUniqueCounter).

Appendice B : Matrice di tracciabilità requisiti → componenti

Requisiti	Componenti principali (package <code>it.collegio</code>)
RF-1 ... RF-6	controllers.LoginController , dao.UserDao , dao.AccessLogDao , dao.SessionsDao , utilities.SessionContext , views.LoginView
RF-7 ... RF-10	controllers.RegistrationController , dao.UserDao , dao.IndirizzoDao , views.RegistrationView
RF-11	controllers.PasswordResetController , views.PassWordView
RF-12 ... RF-18	controllers.ReservationController , dao.ReservationDao , dao.StoricoPrenotazioniDao , dto.ReservationDettaglio , views.ManageRiservationView

RF-19 ... RF-22	<code>controllers.CheckInController</code> , <code>dao.RoomDao</code> , <code>dao.ReservationDao</code> , <code>dao.FatturaDao</code> , <code>views.CheckInView</code>
RF-23 ... RF-26	<code>controllers.CheckOutController</code> , <code>dto.CheckoutPreview</code> , <code>utilities.utility</code> , <code>views.CheckOutView</code>
RF-27 ... RF-30	<code>controllers.FatturaController</code> , <code>dao.FatturaDao</code> , <code>dao.PagamentoDao</code> , <code>dao.MetodoPagamentoDao</code> , <code>dto.FatturaDettaglio</code> , <code>views.FatturaView</code> , <code>views.GestioneFattureView</code>
RF-31	<code>controllers.ManageRoomController</code> , <code>dao.RoomDao</code> , <code>views.ManageRoomView</code>
RF-32	<code>controllers.ManageUserController</code> , <code>dao.UserDao</code> , <code>views.ManageUserView</code>
RF-33	<code>controllers.ManageTenantController</code> , <code>dao.CommittenteDao</code> , <code>dto.CommittenteDettaglio</code> , <code>views.ManageTenantView</code>
RF-34	<code>views.ManageUserView</code> , <code>views.ManageTenantView</code> , <code>views.ManageRoomView</code>
RF-35	<code>dao.AccessLogDao</code> , <code>dto.AccessLogDettaglio</code> , <code>views.LogsView</code>
RF-36	<code>controllers.HomeController</code> , <code>views.HomeView</code>
RNF-1 ... RNF-4	struttura dei package <code>views</code> / <code>controllers</code> / <code>dao</code> / <code>dto</code> / <code>models</code> , <code>utilities.QueryContainer</code>
RNF-5	transazioni in <code>LoginController</code> , <code>CheckInController</code> , <code>CheckOutController</code> , <code>ReservationController</code>
RNF-8, RI-5	<code>dao.DatabaseConnection</code> , <code>config/db.properties</code>
RNF-12	<code>utilities.utility</code> (<code>isValidEmail</code> , <code>validaNumeroTelefono</code>)
RNF-13	<code>views.LoadingView</code> , <code>views.Loading1View</code>

Appendice C : Enumerazioni del dominio

Enum	Valori	Uso
<code>UserStatus</code>	attivo, disattivato, attesa	Stato account utente (RF-2, RF-8)
<code>RoomStatus</code>	disponibile, occupata, manutenzione	Stato camera (RF-19, RF-21, RF-26)
<code>RoomType</code>	singola, doppia, suite	Tipologia camera (RF-31)
<code>BedType</code>	matrimoniale, singolo, king-size	Tipo letto (RF-31)
<code>ReservationStatus</code>	attiva, cancellata, completata	Stato prenotazione (RF-21, RF-26, RF-30)
<code>FatturaStatus</code>	pagato, non pagato, in attesa	Stato fattura (RF-21, RF-26, RF-28, RF-29, RF-30)
<code>Genere</code>	(incluso ALTRO come fallback)	Anagrafica utente (RF-7)

5. Indice

1. Introduzione

- 1.1 Scopo del documento
- 1.2 Scopo del prodotto
- 1.3 Definizioni, acronimi e abbreviazioni
- 1.4 Riferimenti
- 1.5 Descrizione del resto del documento

2. Descrizione generale

- 2.1 Prospettiva del prodotto
- 2.2 Funzioni del prodotto
- 2.3 Caratteristiche utente
- 2.4 Vincoli generali
- 2.5 Presupposti e dipendenze

3. Requisiti specifici

- 3.1 Requisiti funzionali (RF-1 ... RF-36)
 - 3.1.1 Autenticazione e sessione
 - 3.1.2 Prenotazioni
 - 3.1.3 Check-in
 - 3.1.4 Check-out
 - 3.1.5 Fatture e pagamenti
 - 3.1.6 Gestione anagrafiche
 - 3.1.7 Monitoraggio e controllo accessi
- 3.2 Requisiti non funzionali (RNF-1 ... RNF-17)
 - 3.2.1 Architettura e manutenibilità
 - 3.2.2 Integrità dei dati
 - 3.2.3 Sicurezza
 - 3.2.4 Usabilità
 - 3.2.5 Portabilità e vincoli tecnologici
- 3.3 Requisiti di interfaccia (RI-1 ... RI-8)
 - 3.3.1 Interfacce utente
 - 3.3.2 Interfacce software
 - 3.3.3 Interfacce hardware e di comunicazione

4. Appendici

- A. Schema dati di riferimento (MySQL)
- B. Matrice di tracciabilità requisiti → componenti

C. Enumerazioni del dominio

5. Indice